

Varun Mehrishi

Software Development Engineer II, Amazon | Distributed Systems, Payments and Audit Platforms

Hyderabad, India | [linkedin.com/in/vmehrishi](https://www.linkedin.com/in/vmehrishi) | github.com/varunmehrishi

PROFESSIONAL SUMMARY

Software engineer with 7 years at Amazon building event-driven financial systems that audit and pay carrier invoices across tens of billions of dollars of annual transportation spend. Designed and led delivery of a cross-team enrichment service, a dead-letter-queue platform adopted by six teams, and a configurable dispute-detection engine; led five engineers and two interns to launch, and is reviewer of record for six payment and audit services where a single defect costs millions. Owns production end to end: on-call for seven years, incident lead for Sev2 events, root causes and fixes shipped under change control. Java, AWS (Lambda, SQS, DynamoDB, S3, CDK), Spark and Rust.

CORE COMPETENCIES

Architecture: Event-driven microservices, asynchronous pipelines, idempotency and versioning contracts, optimistic locking, rate limiting, backward-compatible schema evolution, cost modelling

Technical leadership: System design ownership, decomposition and delegation to SDE1s, design and code review across six services, intern mentoring, cross-team alignment with partner engineering and product owners

Operations: On-call and incident command, root-cause analysis, DLQ redrive and recovery tooling, Sev2 mitigation under change management, monitoring and alarm design, capacity and peak planning

WORK EXPERIENCE

Amazon, Transportation Financial Systems

Nov 2019 – Present

Hyderabad, India. Systems that audit, dispute and pay carrier invoices across NA, EU and FE.

Software Development Engineer II

Jul 2022 – Present

- Designed the end-to-end architecture of Meghnad, a shared enrichment service that fills missing shipment scan data from partner APIs and web sources for an org-wide audit-automation program; led three engineers to launch in Apr 2026. Raised shipment scan-data coverage from **98% to 99.8%**; no DLQ or latency incidents since launch.
- Built its asynchronous pipeline (SNS/SQS, bounded Parquet batches, cross-account S3, Spark on EMR) with distributed rate limiting to partner quotas (1,395 and 27 TPS), retries and health checks within a 2-hour SLA; prevented re-enrichment loops with rule-based timestamp checks. Shipped a version-based decision-rollback control across three services in **three days** to unblock launch.
- Led Charge Type Payments to **100% carrier coverage**: owned operational readiness with four partner teams, grew EU from 15 to 130+ carriers and launched 230+ NA carriers with zero launch incidents; led two engineers on the final aggregation layer, replacing contention-prone optimistic locking ($n(n+1)/2$ writes) with FIFO serialization by audit-item identity. Global launch Jul 2026, no regressions.
- Enriched payment notifications with invoiced transaction details so billing consumers decide without fan-out calls to upstream APIs; in partner design reviews, blocked aggregation on paid amounts that would have miscounted direct short payments.
- Co-led mitigation of a duplicate-authorization incident with **\$2.5M** in carrier payments stuck: sized the blast radius with a warehouse query, replaced last-write-wins amount merging with per-transaction summation plus tests, and ran a staged replay (1, 10, all) under change management. Funds released in five days.
- Designed Configurable Property Dispute Detection: a comparator abstraction with structured comparison results and a YAML model with polymorphic tags, so new dispute types ship as configuration; extended to imports and middle-mile freight with volume and container-type comparators.
- Fixed a **\$17M** duplicate dispute-resolution defect with an order-independent equivalence hash (murmur3 over sorted attributes, activation-dated keys for schema evolution, dual-path compatibility).
- Compressed payment records in DynamoDB, cutting storage cost **70%** and letting services absorb 4x transaction volume.

- Built reusable recovery CLIs for records exceeding the Firehose 1 MB limit (dry run, bounded concurrency, idempotent ETag-verified writes); recovered 1,113 records from 1,205 DLQ messages with zero failures. Wrote an experimental Rust indexer (Smithy, Lambda, API Gateway, RDS Serverless) over six months of notifications, **p90 100 ms at 500 TPS**, used to extract identifiers for 200K containers.
- Design and code reviewer of record for six payment and audit services moving tens of billions of dollars a year, where one defect costs millions: **137 reviews in 2025** and 95 in H1 2026, blocking issues such as aggregation on paid amounts (miscounts short payments), unsafe DynamoDB update assumptions and lossy retry classification before release. Mentored two interns (one hired) who delivered a snapshot-based regression framework adopted by three services.
- Incident command: led recovery from a cache outage that cascaded into JVM heap exhaustion (traffic shaping, host replacement, causal RCA) and from an EU traffic surge (fleet 8 to 40 hosts, 1.37M-message backlog drained); found two Sev2 root causes while off-rotation (5 GB S3 object limit, 80M-message backlog from quadruplicated uploads).

Software Development Engineer

Nov 2019 – Jun 2022

- Primary author of TFS Helix, the dead-letter-queue platform for the organization's event-driven services: prevents message loss, surfaces stack traces, publishes metrics, cuts tickets and redrives after fixes. Java 11, jOOQ, AWS Lambda, Multi-AZ MySQL with RDS Proxy, SQS, S3, CDK. Adopted by **six teams** including the first team outside the organization; still the shared platform in 2026 (later migrated to JDK 21, async SDK v2 and an MCP server).
- Built reusable alarm constructs in the shared CDK monitoring library and rolled DLQ alarms to four services; regionalized ticket de-duplication after a production learning.
- Refactored Payment Authorization rule evaluation into generic applicability criteria and deciders so new payment laws became declarative additions instead of code forks.
- Month-end-close captain for auditing (Q3 2021 to Q3 2022): resolved **35 finance-critical tickets** with zero escalations; during on-call reduced the open ticket backlog from 42 to 14 and drained a 1M-message resolution backlog using measured throughput.
- Enriched the invoice-search UI across three data sources and onboarded India carriers with 189 GST translation codes and tax attributes; verified the EU launch of the invoice platform with same-day, log-driven root causes.

Microsoft Consulting Services, Associate Project Manager, Hyderabad

Jul 2019 – Nov 2019

PROJECTS

rdb-parser (Rust, MIT)

2024

- Parser-combinator reader for the Redis RDB binary format using winnow: length encodings, integer-encoded strings, intsets, TTL entries; hex-literal fixtures and unit tests.
github.com/varunmehrishi/rdb-parser

Hackattic solutions (Rust)

2023 – 2025

- Ten systems challenges behind one trait-based async runner (reqwest, tokio, tracing): proof-of-work search with rayon, DTMF decoding with FFT, SSL, script hashing, WebSocket and Redis dump parsing.
github.com/varunmehrishi/Hackattic

SKILLS

Languages: Java, TypeScript, Scala, Python, Rust, SQL, Bash

AWS: Lambda, SQS, SNS, DynamoDB, S3, Kinesis Data Firehose, Step Functions, EMR, RDS (MySQL, Aurora Serverless), API Gateway, CloudWatch, IAM, CDK v2, Athena, Redshift

Frameworks and tools: Smithy and Coral service frameworks, Guice, Dagger, MapStruct, jOOQ, JUnit, Mockito, Apache Spark, Elasticsearch, React, Vite, Git, CI/CD pipelines, MCP servers

Practices: System design, API and schema design, event-driven architecture, idempotency, observability and alarming, load and regression testing, on-call and incident management, code review, mentoring

EDUCATION

Manipal University Jaipur, B.Tech in Computer Science and Engineering, CGPA 9.36/10

2015 – 2019